SIMATS ENGINEERING

CO1-ASSESSMENT TOOL 2 – DATA INTERPRETATION

Student Name: Bemanaveni Manish
Register Number:192411149
Course Name: Internet Programming
Course Code: CSA4308

1. HTTP Response Analysis

Given Data

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

1.1 Incorrect or Invalid Fields

On inspecting the header field by field, the Date, Content-Type, and Connection fields are syntactically valid and well-formed. The single invalid field is Content-Length: -150. The HTTP specification defines Content-Length as a non-negative integer representing the number of bytes in the message body; a negative value such as -150 is not a legal value for this field and violates RFC 9110.

1.2 Analysis of the Content-Length Issue and Its Impact

Content-Length tells the receiving client exactly how many bytes of body data to expect after the headers, so the client knows when the response has finished arriving, especially over a persistent (keep-alive) connection where the socket is not closed after each response. A negative value is meaningless as a byte count, so a compliant client or intermediary (proxy, browser, load balancer) cannot correctly determine where the response body ends.

The practical impact includes: the browser may reject the response outright as malformed; on a keep-alive connection the client may wait indefinitely for bytes that will never arrive, or conversely truncate/misread the body, corrupting the page content; downstream caches and proxies may refuse to cache or forward the response; and in the worst case this class of error is associated with request/response smuggling vulnerabilities, since disagreement between server and intermediary about body length can be exploited.

1.3 Does the Status Code Match the Response Correctness?

The status line reports 200 OK, which asserts that the request succeeded and a valid representation is being returned. This is inconsistent with the malformed Content-Length value. A response cannot simultaneously be fully successful and contain a header that violates the protocol specification. Strictly speaking, the server

has generated an internally inconsistent response: the semantic status (200 OK) promises a good response, but the syntactic content (a negative Content-Length) is invalid, so the response as a whole cannot be trusted or correctly processed by the client.

1.4 Suggested Corrections

1. Replace Content-Length: -150 with the actual, accurate, non-negative byte length of the response body, e.g., Content-Length: 150 (calculated from the real body size).
2. If the exact body length cannot be known in advance (e.g., for dynamically generated or streamed content), remove Content-Length entirely and instead use Transfer-Encoding: chunked, which lets the server send the body in chunks without pre-declaring its total size.
3. Add server-side validation before the response is dispatched so that a negative or non-numeric Content-Length can never be sent to a client.
4. Log and monitor such malformed headers on the server, since they usually indicate a bug in the code that calculates body size (for example, an unhandled subtraction producing a negative number).

2. DOM Structure Analysis

Given Data

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div> <p>Hello World
  </div>
  </body>
</html>
```

2.1 Missing or Improperly Closed Elements

The element that wraps "Hello World" is opened but never explicitly closed with a tag. Instead, the markup jumps directly to , so the closing tag for appears before a closing tag for exists. This breaks the expected nesting order, where the most recently opened element must be the next one closed.

2.2 How the Browser Will Interpret This Structure

HTML parsers are built to be fault-tolerant using well-defined error-recovery rules (the HTML5 parsing algorithm), so the browser does not simply fail. Because is defined as an element that automatically closes when certain other tags are encountered, the browser's parser will treat the unclosed as implicitly closed at the point where appears. The resulting DOM effectively becomes Hello World , with the browser silently inserting the missing tag into its internal DOM tree rather than displaying an error.

2.3 Impact on Rendering and Layout

In this specific case, because the browser's auto-closing rule produces the same structure the developer likely intended, the visible output on screen is usually unaffected — "Hello World" still renders as a paragraph inside the div. However, relying on this auto-correction is risky: if further nested elements had been placed after <p> before the intended closing point, the browser's auto-closing behaviour could pull those elements out of the paragraph unexpectedly, producing broken nesting, inconsistent spacing/margins (since <p> has default block-level margins), and unpredictable results across different browsers or when the page is processed by tools that are stricter than a browser (such as an XML/XHTML parser, which would fail outright rather than auto-correct).

2.4 Corrections to Make the DOM Valid

The corrected markup explicitly closes every element in the reverse order it was opened:

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World</p>
    </div>
  </body>
</html>
```

Explicitly closing <p> before removes any dependency on the browser's error-recovery algorithm, ensures the same, predictable DOM tree is produced in every browser, and keeps the markup valid against the HTML specification (and compatible with XHTML/XML tooling, which does not perform auto-correction).

3. GET vs POST in Login System

Given Data

GET: /login?user=admin&pass=1234 – data visible in the URL – Low security level.

POST: /login – data hidden in the request body – Moderate security level.

3.1 Differences in Data Transmission

With GET, form data is appended to the URL as a query string (?user=admin&pass=1234), meaning the credentials travel as plain, visible text in the request line itself. With POST, the same data is placed inside the body of the HTTP request, separate from the URL, so it does not appear in the address bar, browser history, bookmarks, or typically in server access logs, which usually record only the URL and method, not the body.

3.2 Which Method Is More Suitable for Login, and Why

POST is the more suitable method for a login system. Login submissions are non-idempotent, state-changing operations (they can trigger session creation, failed-attempt counters, or account lockouts), which aligns with POST's semantics, whereas GET is meant for idempotent, side-effect-free retrieval and is also cacheable — a serious problem if a browser or proxy were to cache a URL that contains a password. Keeping the credentials in the body also reduces (though does not by itself guarantee) exposure of sensitive data.

3.3 Security Risks of Using GET for Authentication

- Credentials appear in plain text in the browser's address bar and are stored in browser history, making them recoverable by anyone with access to the device.
- URLs are commonly logged by web servers, proxies, load balancers, and analytics tools, so the username and password can end up written to persistent log files.
- URLs can be cached by browsers and intermediate proxies, and may be leaked via the HTTP Referer header if the login page links to external resources.
- URLs are easily shared accidentally (copy-pasted, bookmarked, sent in chat), exposing the credentials to unintended recipients.
- GET requests are more exposed to shoulder-surfing and to being captured in browser autocomplete suggestions.

3.4 Best Practices for Secure Login Implementation

1. Always submit login forms using method="POST", never GET, so credentials are not exposed in the URL.
2. Serve the entire login flow over HTTPS (TLS), since POST alone does not encrypt the body — without TLS, credentials in the POST body can still be intercepted in transit.
3. Hash and salt passwords on the server (e.g., using bcrypt or Argon2) so plaintext passwords are never stored, and never echo the password back in responses or logs.
4. Implement additional protections such as rate limiting or account lockout after repeated failed attempts, CSRF tokens on the login form, and secure, HttpOnly, SameSite cookies for session management after a successful login.

4. HTML Code Inspection

Given Data

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
```

```
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

4.1 Missing Closing Tags

Two elements are opened without an explicit closing tag: <h1>Welcome is never closed with </h1>, and This is a paragraph is never closed with . Both should have an explicit end tag before the next element begins.

4.2 Issues with the <img> Tag

The <img> tag itself is a void (self-closing) element in HTML, so the absence of a closing tag is not an error in standard HTML5 syntax. However, the tag as written is missing the alt attribute, which is required for accessibility (screen readers rely on it to describe the image to visually impaired users) and is also displayed as fallback text if the image fails to load. Best practice is also to specify width and height attributes to prevent layout shift while the image loads.

4.3 How Browsers Will Handle These Errors

As with the DOM example above, HTML browsers apply built-in error-recovery rules rather than failing to render the page. Because <h1> and are block-level elements that implicitly close when another block-level element begins, the browser will auto-close <h1> when it encounters , and will auto-close when it encounters the <img> tag (or the following </body>). The missing alt attribute does not stop rendering at all; the browser simply displays the image without descriptive fallback text, silently degrading accessibility rather than raising a visible error.

4.4 Corrected Version of the Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph</p>
  <img src="image.jpg" alt="Descriptive text about the image" width="600" height="400">
</body>
</html>
```

This version explicitly closes every element that requires a closing tag, and adds the alt attribute (plus explicit dimensions) to the <img> tag so the page is valid, accessible, and renders predictably in every browser.

5. Browser-Server Communication Flow

Given Sequence

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes the request
5. Response is returned
6. Browser renders the page

5.1 Interpretation of Each Step

- Step 1 – URL entry: The browser parses the URL to identify the scheme (http/https), the hostname, the port, the path, and any query parameters, and determines what kind of request needs to be built.
- Step 2 – DNS lookup: The browser resolves the human-readable hostname (e.g., www.example.com) into a numeric IP address, checking the browser cache, OS cache, router, and ultimately recursive/authoritative DNS servers if the name is not already cached.
- Step 3 – HTTP request: Once the IP address is known, the browser establishes a TCP connection (and a TLS handshake if HTTPS is used) to the server and sends an HTTP request containing the method, headers, and, if applicable, a body.
- Step 4 – Server processing: The web server (and any application layer, database queries, or business logic behind it) receives the request, processes it, and prepares a response, which may involve reading files, querying a database, or running server-side code.
- Step 5 – Response returned: The server sends back an HTTP response consisting of a status line, headers, and typically a body (HTML, JSON, an image, etc.), which travels back over the same connection.
- Step 6 – Rendering: The browser parses the received HTML into the DOM, fetches any additional linked resources (CSS, JavaScript, images), builds the render tree, and paints the final page on screen.

5.2 Where Delays Can Occur

- DNS resolution delay: if the domain is not cached anywhere, the recursive lookup across multiple DNS servers can add noticeable latency.
- Connection setup delay: the TCP handshake, and especially the TLS handshake for HTTPS, adds one or more network round trips before any data is exchanged.
- Network transmission delay: physical distance between client and server, network congestion, and limited bandwidth all add latency to both the request and response.
- Server processing delay: slow database queries, unoptimised server-side code, or an overloaded server can significantly delay step 4.

- Rendering delay: large HTML/CSS/JavaScript payloads, render-blocking scripts and stylesheets, and many additional resource requests (images, fonts) can slow down step 6 even after the response has fully arrived.

5.3 Role of DNS in the Process

DNS acts as the internet's directory service: it translates the memorable domain name typed by the user into the numeric IP address that computers actually use to route network traffic. Without this translation step, the browser would have no way to know which physical server to connect to. Because DNS lookups happen before any HTTP request can be sent, DNS sits directly on the critical path of every single page load, and caching at multiple levels (browser, OS, router, ISP resolver) exists specifically to avoid repeating this lookup unnecessarily.

5.4 Suggested Optimizations to Improve Performance

1. Reduce DNS latency using DNS caching, DNS prefetching (<link rel="dns-prefetch">), and reliable, low-latency DNS resolvers, and minimise the number of distinct third-party domains a page depends on.
2. Reduce connection setup cost by using HTTP/2 or HTTP/3 (which support multiplexing and reduce handshake overhead), enabling persistent/keep-alive connections, and using TLS session resumption.
3. Reduce server processing time through database query optimisation, server-side caching (e.g., caching rendered pages or query results), and load balancing across multiple servers.
4. Reduce data transfer time by compressing responses (gzip/Brotli), using a Content Delivery Network (CDN) to serve resources from a location physically closer to the user, and minifying HTML, CSS, and JavaScript.
5. Speed up rendering by deferring or asynchronously loading non-critical JavaScript, minimising render-blocking CSS, optimising and lazy-loading images, and reducing the overall number of HTTP requests needed to display the page.

Overall Conclusion

Across all five scenarios, the recurring theme is that strict adherence to web standards — correctly formed HTTP headers, properly nested and closed HTML/DOM elements, appropriate use of HTTP methods, and an efficient browser-server communication pipeline — directly determines the reliability, security, and performance of a web application. A single malformed field such as a negative Content-Length, an unclosed <p> tag, or the use of GET instead of POST for login credentials may not always cause a visible failure, because browsers are highly fault-tolerant, but each one introduces real risks: parsing ambiguity, security exposure, degraded accessibility, or unnecessary latency. Consistently validating markup, choosing HTTP methods based on their intended semantics, and optimising each stage of the request-response cycle are therefore essential practices for building standards-compliant, secure, and performant web applications.